Computer Architecture

Dynamic Scheduling II

Special thanks to Prof. Daniel Sorin, Prof. Milo Martin, Prof. Amir Roth, Prof. Mark Hill, and Prof. David Wood. These slides heavily borrow theirs.

Dynamic Scheduling II

- So far: dynamic scheduling (out-of-order execution)
  - Scoreboard
  - Tomasulo's algorithm
  - Register renaming: removing artificial dependencies (WAR/WAW)

- **Now**: out-of-order execution + precise state

- **Advanced topic**: dynamic load scheduling

Superscalar + OoO + Speculation

- Superscalar + out-of-order + speculation
  - Three concepts that work well (best?) when used together
  - **CPI >= 1?**
    - Overcome with superscalar
  - **Superscalar increases hazards?**
    - Overcome with dynamic scheduling
  - **RAW dependencies still a problem?**
    - Overcome with a large instruction window
  - **Branches a problem for filling large window?**
    - Overcome with speculation

Speculation and Precise Interrupts

- Q: Why are we discussing these together?
  - Sequential (von Neumann) semantics for **interrupts**
    - All instructions **before** interrupt should be complete
    - All instructions **after** interrupt should look as if never started
    - We also want the same thing for a mis-predicted branch
  - What makes precise interrupts hard?
    - Out-of-order completion ⇒ must undo post-interrupt writebacks

- A: With out-of-order pipe, **precise interrupts** and **mis-speculation recovery** are the same problem
  - Same solution
  - Implement ability to abort & restart **at every instruction**
  - Called "**precise state**"

The Problem with Precise State

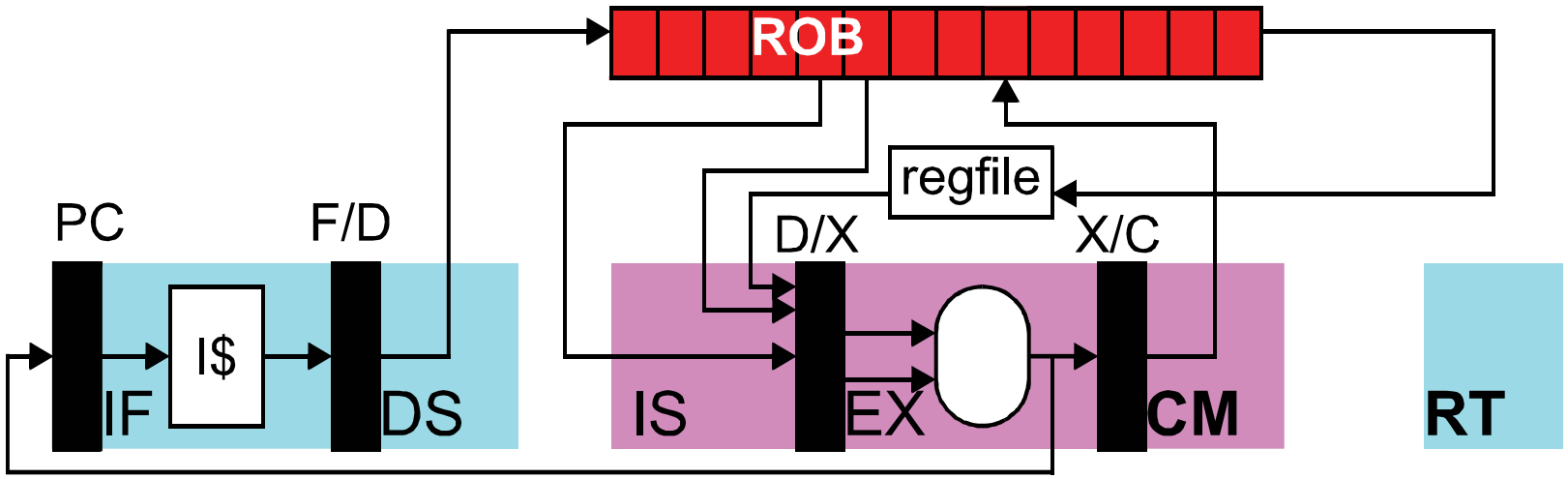
- Problem is in the **write-back stage (WB)**
  - Mixes two things together that should be separate

  (1) Broadcasts values to RS (forwards to other instructions)
    - OK for this to be **out-of-order**

  (2) Writes values to registers
    - Would like this to **be in-order**

- Solution
  - **Split write-back into two separate stages**

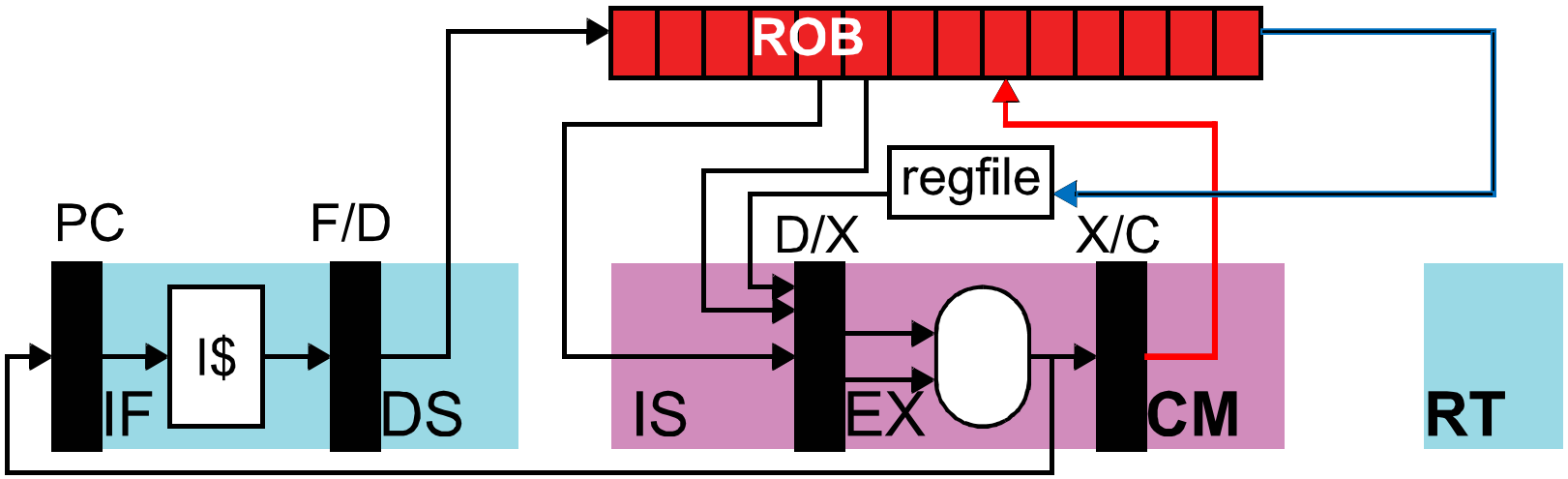
Reorder Buffer (ROB)



- **Instruction buffer ⇒ re-order buffer (ROB)**

- Split WB into two stages
  - **Complete** and **Retire**

Reorder Buffer (ROB)



- **CM (complete)**
  - Completed result values written to ROB **out-of-order**

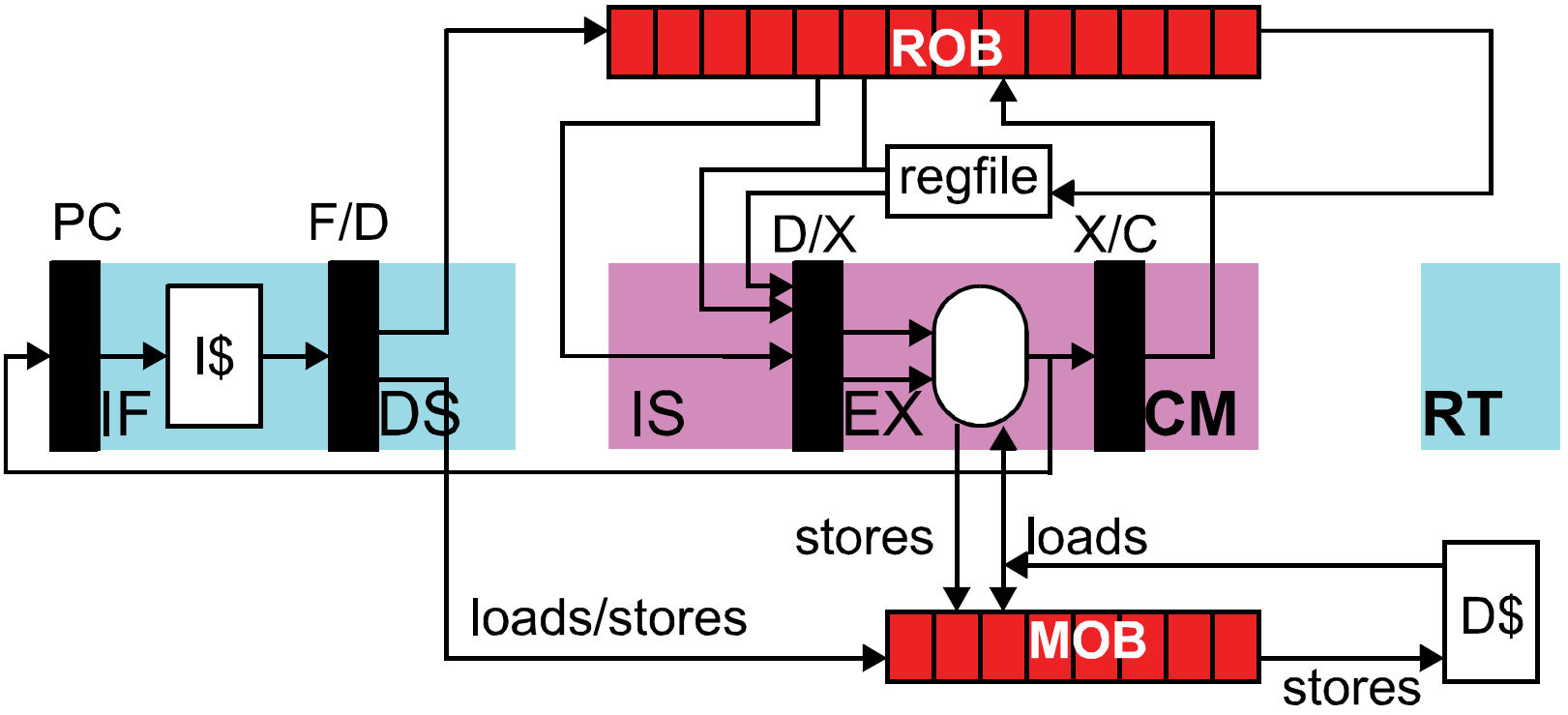
- **RT (retire, sometimes called "commit"/"graduate")**
  - ROB writes results to register file/D$ **in-order**
  - In-order stage ⇒ hazards result in stalls

Memory Ordering Buffer (MOB)

- ROB makes register writes in-order, but what about stores?
  - Same as before (i.e., to D$ in MEM stage)?
    - Bad idea! imprecise memory worse than imprecise registers
    - Must use same trick for stores

- **Memory Ordering Buffer (MOB)**
  - a.k.a. store buffer, store queue, load/store queue (LSQ)
  - Completed (but not retired) stores write to MOB
  - To retire store, write head of MOB to D$
  - Loads look at MOB and D$ in parallel
    - Forward from MOB if matching store (i.e. to same address)

ROB + MOB



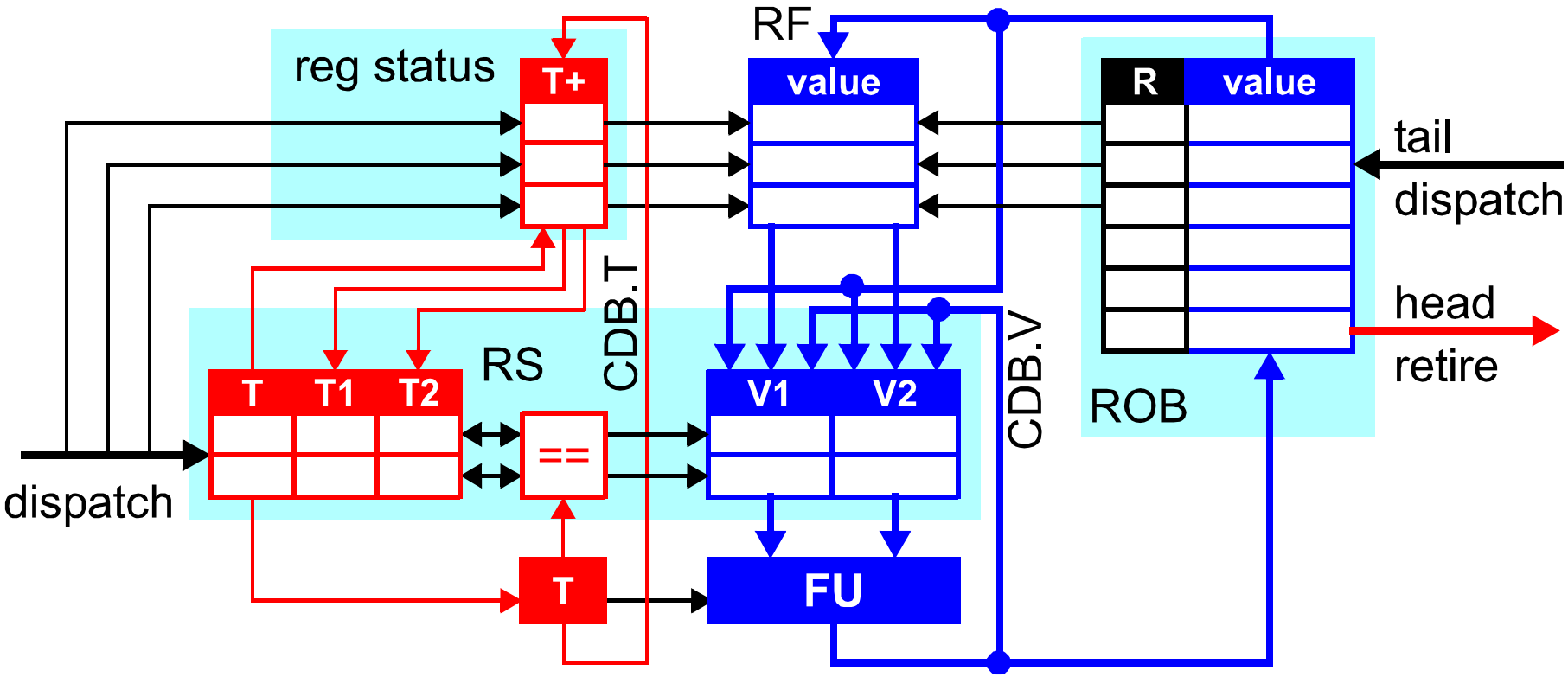
- Intel's P6 architecture
  - Almost realistic with the exception of some gross simplifications ☺

Tomasulo + ROB

- Add ROB to Tomasulo's algorithm
  - Separate ROB and RS are called P6-style

- Our example: Simple-P6 (Pentium Pro)
  - Separate ROB and RS
  - Same RS organization as before
    - 1 ALU, 1 load, 1 store, 2 (3-cycle) FP

P6-style Organization



- Instruction fields and ready bits
- Tags
- Values

P6 Data Structures

- RS are the same as before
  - ROB
    - **head, tail:** to keep sequential order
    - **R:** output register of instruction, V: output value of instruction

- Tags are different
  - was: RS# → now: ROB#

- T register status table is different
  - **T+:** tag + "ready-in-ROB" bit
  - tag == 0 ⇒ result ready in register file
  - tag != 0 ⇒ result not ready
  - tag != 0 + ⇒ result ready in ROB

P6 Pipeline

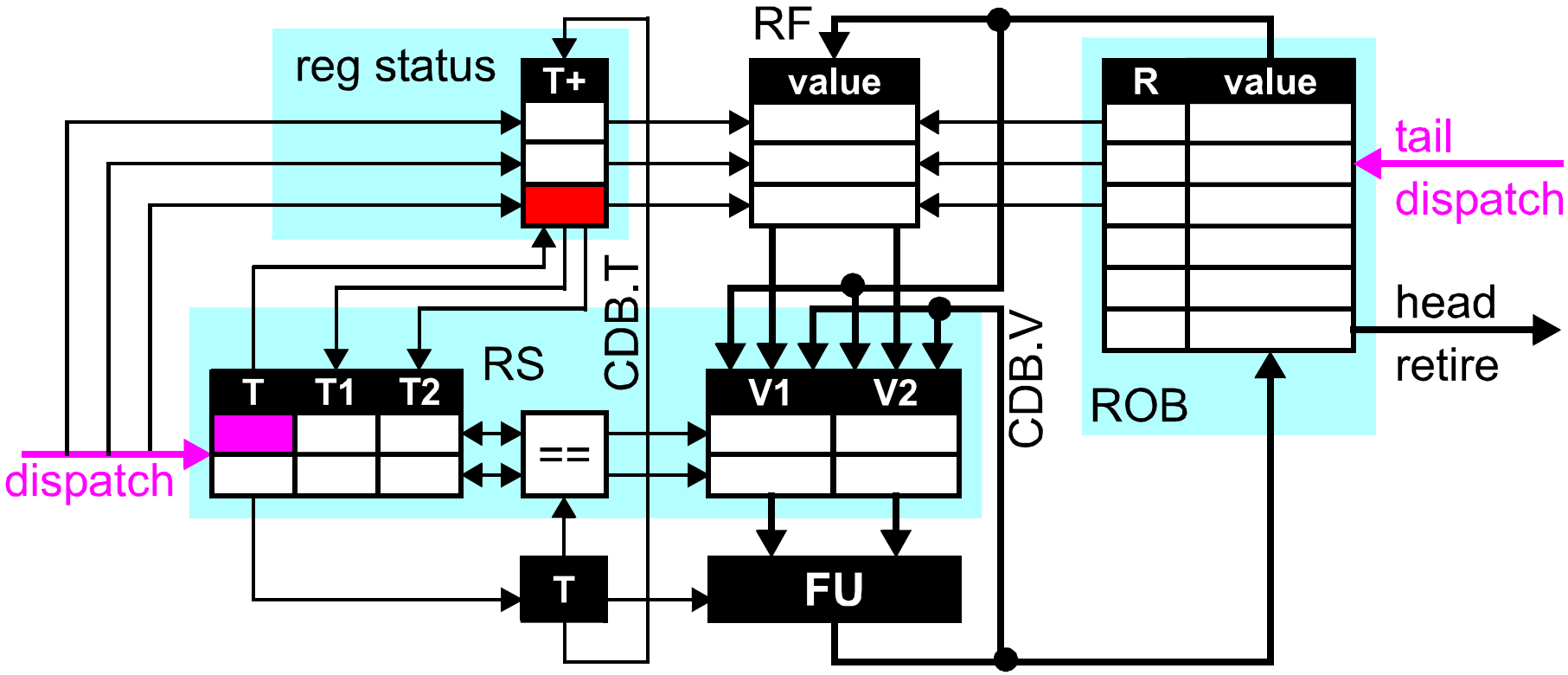
- New pipeline structure: IF, **DS**, IS, **EX, CM, RT**
- **DS (dispatch)**
  - (RS/ROB/MOB full) ? (stall) :
    1. Allocate RS/ROB/MOB entries
    2. Set RS tag to ROB#
    3. Set register status entry to ROB# with "ready-in-ROB" bit off
    4. Copy ready registers into RS
- **IS (issue)**
  - (If no RAW) ? (execute) : (wait, monitor CDB)
- **EX (execute)**
  - Free RS entry
    - Used to be done at WB
    - Can be earlier now because RS# are not tags

P6 Pipeline

- **CM (complete)**
  - (CDB not available) ? (stall) :
    1. Broadcast value,tag on CDB
    2. Write value into ROB entry indicated by RS tag
    3. Mark ROB entry complete
    4. Mark register status entry "ready-in-ROB" bit (+)

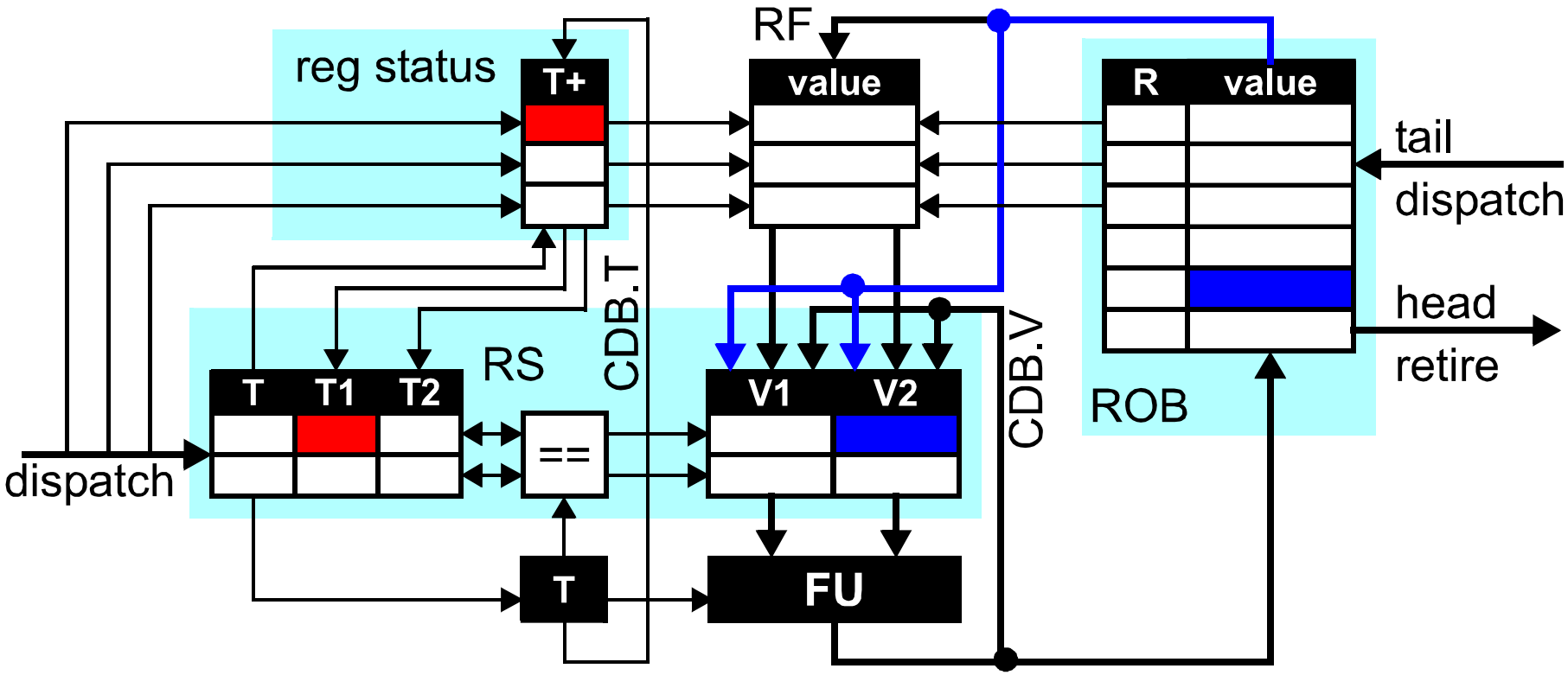
- **RT (retire, commit, graduate)**
  - (ROB head not complete) ? (stall) :
    1. Handle any exceptions
    2. Write ROB head result to register file
    3. If store, then write MOB head to D$
    4. Free ROB/MOB entries

P6: Dispatch (DS) part 1



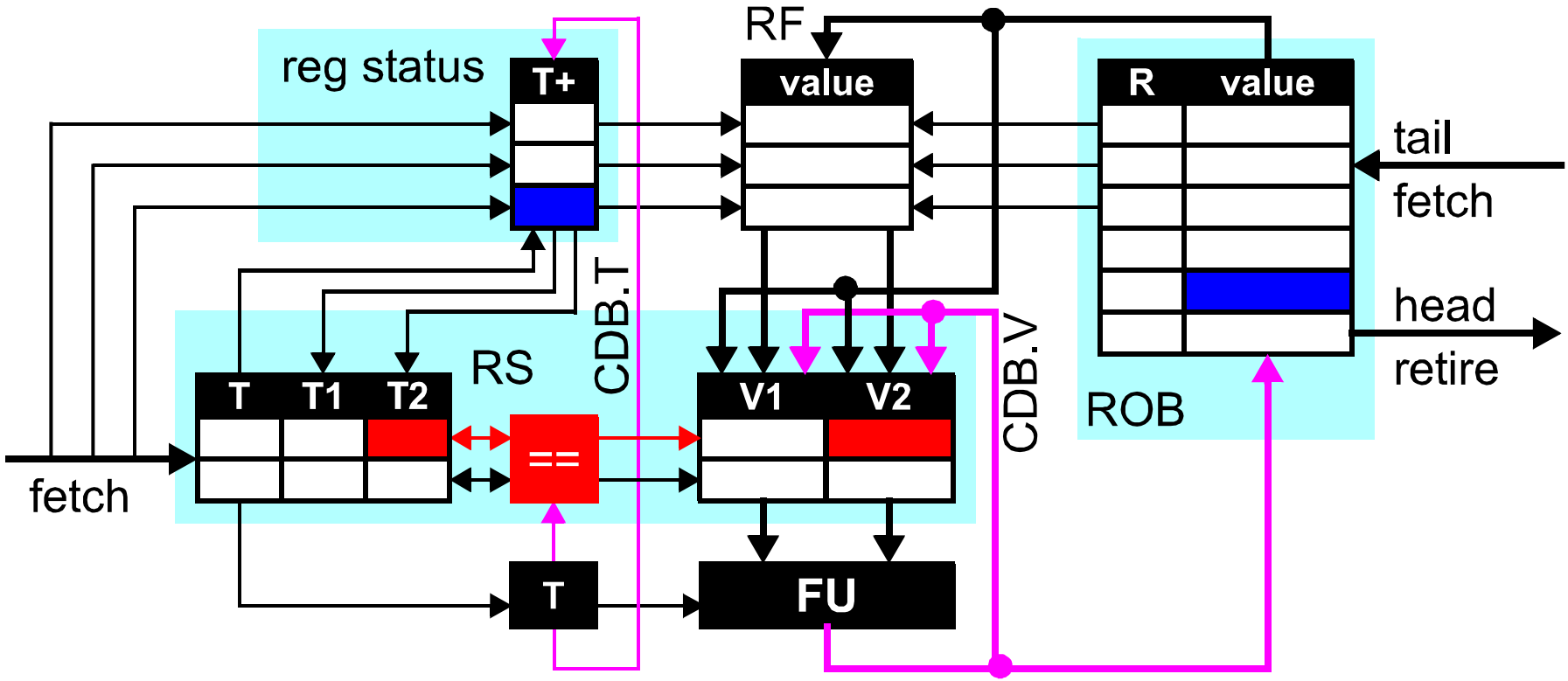
- Stall if RS or ROB/MOB is full
  - Allocate RS+ROB entries (assign ROB# to RS output tag)
  - Set register status entry to ROB# and "ready-in-ROB" bit to 0

P6: Dispatch (DS) part 2



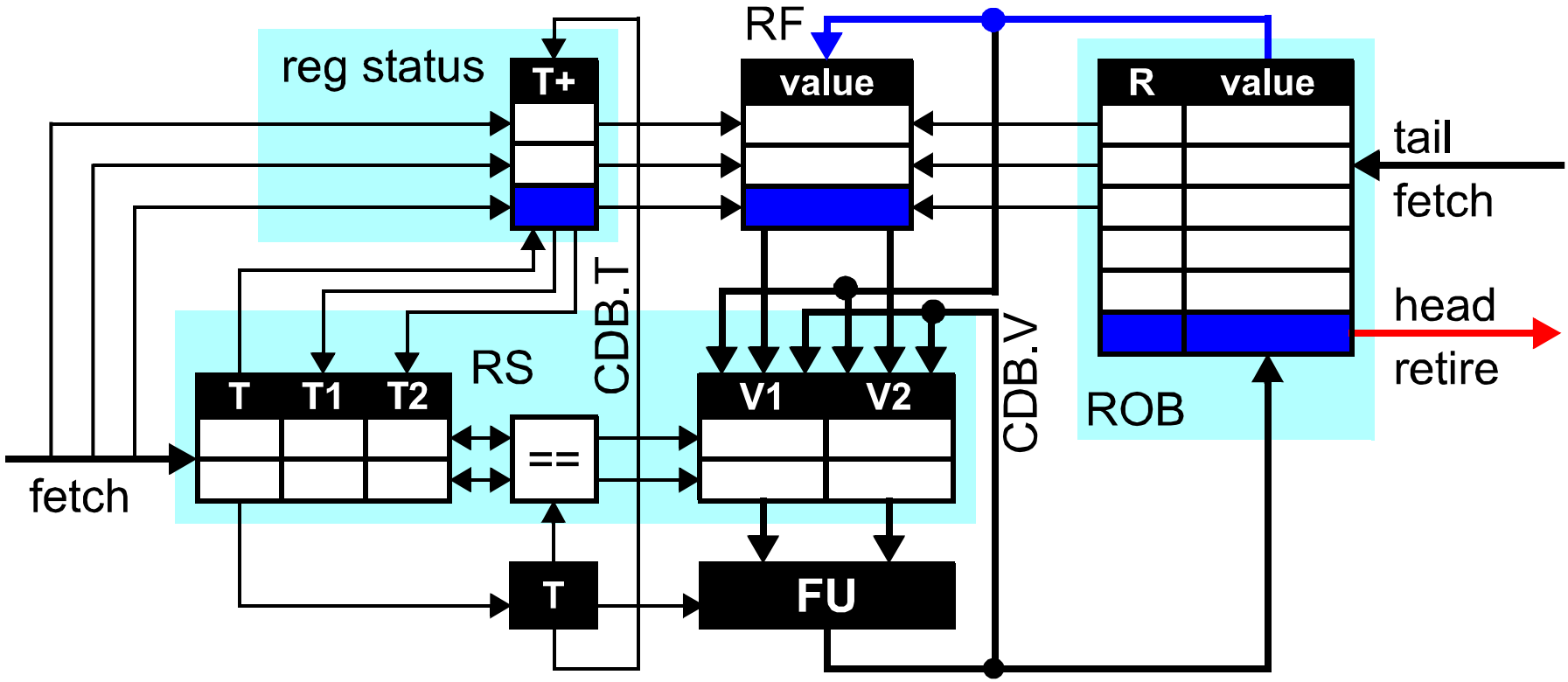
- Read tags for register inputs from register status table
  - if tag==0: copy value from RF (not shown)
  - if tag!=0: copy tag to RS
  - if tag!=0 +: copy value from ROB

P6: Complete (CM)



- Wait for CDB
  - Broadcast <result,tag> on CDB
  - Write result into ROB, set reg. status "ready-in-ROB" bit (+)
  - Match tags, write CDB.V into RS of dependent instructions

P6: Retire (RT)



- Stall until instruction at ROB head has completed
  - Write ROB head result to reg-file (D$ if store), clear reg. status entry
  - Free ROB entry

P6-style Data Structures

	ROB + MOB							
h /t	#	Instruction	R	V	addr	IS	EX	WB
	1	ldf f0,X(r1)						
	2	mulf f4,f0,f2						
	3	stf f4,Z(r1)						
	4	addi r1,r1,#4						
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Reg. Status	
Reg.	T
f0	
f2	
f4	
r1	

CDB	
Reg.	T
r1	

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	No						
3	store	No						
4	FP1	No						
5	FP2	No						

P6 Example: Cycle 1

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
ht	1	ldf f0,X(r1)	f0		&X[0]			
	2	mulf f4,f0,f2						
	3	stf f4,Z(r1)						
	4	addi r1,r1,#4						
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Reg. Status	
Reg	T+
f0	ROB#1
f2	
f4	
r1	

CDB	
V	T

Before ROB, this was RS#2

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	Yes	ldf	ROB#1		REG[r1]		
3	store	No						
4	FP1	No						
5	FP2	No						

Allocate RS
Set reg. status

P6 Example: Cycle 2

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
h	1	ldf f0,X(r1)	f0		&X[0]	2		
t	2	mulf f4,f0,f2	f4					
	3	stf f4,Z(r1)						
	4	addi r1,r1,#4						
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Reg. Status	
Reg	T+
f0	ROB#1
f2	
f4	ROB#2
r1	

CDB	
V	T

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	Yes	ldf	ROB#1		REG[r1]		
3	store	No						
4	FP1	Yes	mulf	ROB#2		REG[f2]	ROB#1	
5	FP2	No						

Allocate ROB,
Allocate RS,
Set reg. status

P6 Example: Cycle 3

	ROB + MOB							
h/ t	#	Instruction	R	V	addr	IS	EX	CM
h	1	ldf f0,X(r1)	f0		&X[0]	2	3	
	2	mulf f4,f0,f2	f4					
t	3	stf f4,Z(r1)			&Z[0]			
	4	addi r1,r1,#4						
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Reg. Status	
Reg	T+
f0	ROB#1
f2	
f4	ROB#2
r1	

CDB	
V	T

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	No						
3	store	Yes	stf	ROB#3		REG[r1]	ROB#2	
4	FP1	Yes	mulf	ROB#2		REG[f2]	ROB#1	
5	FP2	No						

Stall on RAW

Free

Allocate

P6 Example: Cycle 4

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
h	1	ldf f0,X(r1)	f0	[f0]	&X[0]	2	3	4
	2	mulf f4,f0,f2	f4			4		
	3	stf f4,Z(r1)			&Z[0]			
t	4	addi r1,r1,#4	r1					
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Reg. Status	
Reg	T+
f0	ROB#1+
f2	
f4	ROB#2
r1	ROB#4

CDB	
V	T
[f0]	ROB#1

ldf finished
1. Write result to ROB
2. CDB broadcast
3. Set ready-in-ROB bit

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	Yes	addi	ROB#4	REG[r1]			
2	load	No						
3	store	Yes	stf	ROB#3		REG[r1]	ROB#2	
4	FP1	Yes	mulf	ROB#2	CDB.V	REG[f2]	~~ROB#1~~	
5	FP2	No						

Allocate

f0 ready
Get from CDB

P6 Example: Cycle 5

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
	1	ldf f0,X(r1)	f0	[f0]	&X[0]	2	3	4
h	2	mulf f4,f0,f2	f4			4	5	
	3	stf f4,Z(r1)			&Z[0]			
	4	addi r1,r1,#4	r1			5		
t	5	ldf f0,X(r1)	f0					
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Reg. Status	
Reg	T+
f0	ROB#5
f2	
f4	ROB#2
r1	ROB#4

CDB	
V	T

Retire, write ROB result
In regfile

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	Yes	addi	ROB#4	REG[r1]			
2	load	Yes	ldf	ROB#5				ROB#4
3	store	Yes	stf	ROB#3		REG[r1]	ROB#2	
4	FP1	No						
5	FP2	No						

Allocate

Free
*unit Pipelined

P6 Example: Cycle 6

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
	1	ldf f0,X(r1)						
h	2	mulf f4,f0,f2	f4			4	5+	
	3	stf f4,Z(r1)			&Z[0]			
	4	addi r1,r1,#4	r1			5	6	
	5	ldf f0,X(r1)	f0					
t	6	mulf f4,f0,f2	f4					
	7	stf f4,Z(r1)						

Reg. Status	
Reg	T+
f0	ROB#5
f2	
f4	ROB#6
r1	ROB#4

Rename f4

CDB	
V	T

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	Yes	ldf	ROB#5				ROB#4
3	store	Yes	stf	ROB#3		REG[r1]	ROB#2	
4	FP1	No						
5	FP2	Yes	mulf	ROB#6		REG[f2]	ROB#5	

Free

Allocate

P6 Example: Cycle 7

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
	1	ldf f0,X(r1)						
h	2	mulf f4,f0,f2	f4			4	5+	
	3	stf f4,Z(r1)			&Z[0]			
	4	addi r1,r1,#4	r1	[r1]		5	6	7
	5	ldf f0,X(r1)	f0		&X[1]	7		
t	6	mulf f4,f0,f2	f4					
	7	stf f4,Z(r1)						

Reg. Status	
Reg	T+
f0	ROB#5
f2	
f4	ROB#6
r1	ROB#4+

CDB	
V	T
[r1]	ROB#4

addi finished
Write result into ROB, CDB

Stall: DS, no free RS

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	Yes	ldf	ROB#5	CDB.V		~~ROB#4~~	
3	store	Yes	stf	ROB#3		REG[r1]	ROB#2	
4	FP1	No						
5	FP2	Yes	mulf	ROB#6		REG[f2]	ROB#5	

r1 ready
Get from CDB

P6 Example: Cycle 8

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
	1	ldf f0,X(r1)						
h	2	mulf f4,f0,f2	f4	[f4]		4	5+	8
	3	stf f4,Z(r1)			&Z[0]	8		
	4	addi r1,r1,#4	r1	[r1]		5	6	7
	5	ldf f0,X(r1)	f0		&X[1]	7	8	
t	6	mulf f4,f0,f2	f4					
	7	stf f4,Z(r1)						

Reg. Status	
Reg	T+
f0	ROB#5
f2	
f4	ROB#6
r1	ROB#4+

CDB	
V	T
[f4]	ROB#2

Stall RT

Stall: DS, no free RS

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	No						
3	store	Yes	stf	ROB#3	CDB.V	REG[r1]	~~ROB#2~~	
4	FP1	No						
5	FP2	Yes	mulf	ROB#6		REG[f2]	ROB#5	

Free

f4 ready
Get from CDB

P6 Example: Cycle 9

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
	1	ldf f0,X(r1)						
	2	mulf f4,f0,f2	f4	[f4]		4	5+	8
h	3	stf f4,Z(r1)			&Z[0]	8	9	
	4	addi r1,r1,#4	r1	[r1]		5	6	7
	5	ldf f0,X(r1)	f0		&X[1]	7	8	9
	6	mulf f4,f0,f2	f4			9		
t	7	stf f4,Z(r1)			&Z[1]			

Reg. Status	
Reg	T+
f0	ROB#5+
f2	
f4	ROB#6
r1	ROB#4+

CDB	
V	T
[f0]	ROB#5

Retire

Stall RT, not at head

Read from ROB not regfile

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	No						
3	store	Yes	stf	ROB#7		ROB#4.V	ROB#6	
4	FP1	No						
5	FP2	Yes	mulf	ROB#6	CDB.V	REG[f2]	~~ROB#5~~	

Free (ROB#3)
Allocate (ROB#7)
f0 ready
Get from CDB

P6 Example: Cycle 10

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
	1	ldf f0,X(r1)						
	2	mulf f4,f0,f2						
h	3	stf f4,Z(r1)			&Z[0]	8	9	10
	4	addi r1,r1,#4	r1	[r1]		5	6	7
	5	ldf f0,X(r1)	f0		&X[1]	7	8	9
	6	mulf f4,f0,f2	f4			9	10	
t	7	stf f4,Z(r1)			&Z[1]			

Reg. Status	
Reg	T+
f0	ROB#5+
f2	
f4	ROB#6
r1	ROB#4+

Stall RT

CDB	
V	T

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	No						
3	store	Yes	stf	ROB#7	CDB.V	ROB#4.V	ROB#6	
4	FP1	No						
5	FP2	No						

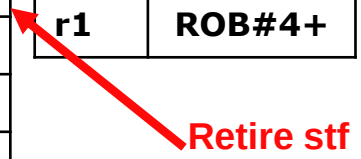
Free

P6 Example: Cycle 11

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
	1	ldf f0,X(r1)						
	2	mulf f4,f0,f2						
h	3	stf f4,Z(r1)			&Z[0]	8	9	10
	4	addi r1,r1,#4	r1	[r1]		5	6	7
	5	ldf f0,X(r1)	f0		&X[1]	7	8	9
	6	mulf f4,f0,f2	f4			9	10	
t	7	stf f4,Z(r1)			&Z[1]			

Reg. Status	
Reg	T+
f0	ROB#5+
f2	
f4	ROB#6
r1	ROB#4+

CDB	
V	T



Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	No						
3	store	Yes	stf	ROB#7	CDB.V	ROB#4.V	ROB#6	
4	FP1	No						
5	FP2	No						

Precise State with ROB

- Point of ROB is maintaining **precise state**
  - Steps:

    1: Wait until condition reaches retire (RT) stage

    2: Clear contents of ROB, RS & register status table

    3: Start over
  - Clear works because it signifies the correct state for start-over
    - 0 in ROB/RS ⇒ entry is empty
    - tag==0 in register status table ⇒ register is written
  - And because register file and D$ writes take place at RT
  - Let's look at example of an interrupt at **stf ...**

P6 Precise State Example: Cycle 11

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
	1	ldf f0,X(r1)						
	2	mulf f4,f0,f2						
h	3	stf f4,Z(r1)			&Z[0]	8	9	10
	4	addi r1,r1,#4	r1	[r1]		5	6	7
	5	ldf f0,X(r1)	f0		&X[1]	7	8	9
	6	mulf f4,f0,f2	f4			9	10	
t	7	stf f4,Z(r1)			&Z[1]			

Reg. Status	
Reg	T+
f0	ROB#5+
f2	
f4	ROB#6
r1	ROB#4+

CDB	
V	T

While trying to retire, stf encounters an interrupt

Must undo …

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	No						
3	store	Yes	stf	ROB#7	CDB.V	ROB#4.V	ROB#6	
4	FP1	No						
5	FP2	No						

P6 Precise State Example: Cycle 12

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
	1	ldf f0,X(r1)						
	2	mulf f4,f0,f2						
	3	stf f4,Z(r1)						
	4	addi r1,r1,#4						
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Reg. Status	
Reg	T+
f0	
f2	
f4	
r1	

CDB	
V	T

Faulting insn at ROB head?
CLEAR EVERYTHING

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	No						
3	store	No						
4	FP1	No						
5	FP2	No						

P6 Precise State Example: Cycle 13

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
	1	ldf f0,X(r1)						
	2	mulf f4,f0,f2						
ht	3	stf f4,Z(r1)			&Z[0]			
	4	addi r1,r1,#4						
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Reg. Status	
Reg	T+
f0	
f2	
f4	
r1	

CDB	
V	T

Start over

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	No						
2	load	No						
3	store	Yes	stf	ROB#3	REG[f4]	REG[r1]		
4	FP1	No						
5	FP2	Yes						

P6 Precise State Example: Cycle 14

	ROB + MOB							
h/t	#	Instruction	R	V	addr	IS	EX	CM
	1	ldf f0,X(r1)						
	2	mulf f4,f0,f2						
h	3	stf f4,Z(r1)			&Z[0]	14		
t	4	addi r1,r1,#4	r1					
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Reg. Status	
Reg	T+
f0	
f2	
f4	
r1	ROB#4

CDB	
V	T

Reservation Stations								
T	FU	Busy	op	T	V1	V2	T1	T2
1	ALU	Yes	addi	ROB#4	REG[r1]			
2	load	No						
3	store	Yes	stf	ROB#3	REG[f4]	REG[r1]		
4	FP1	No						
5	FP2	Yes						

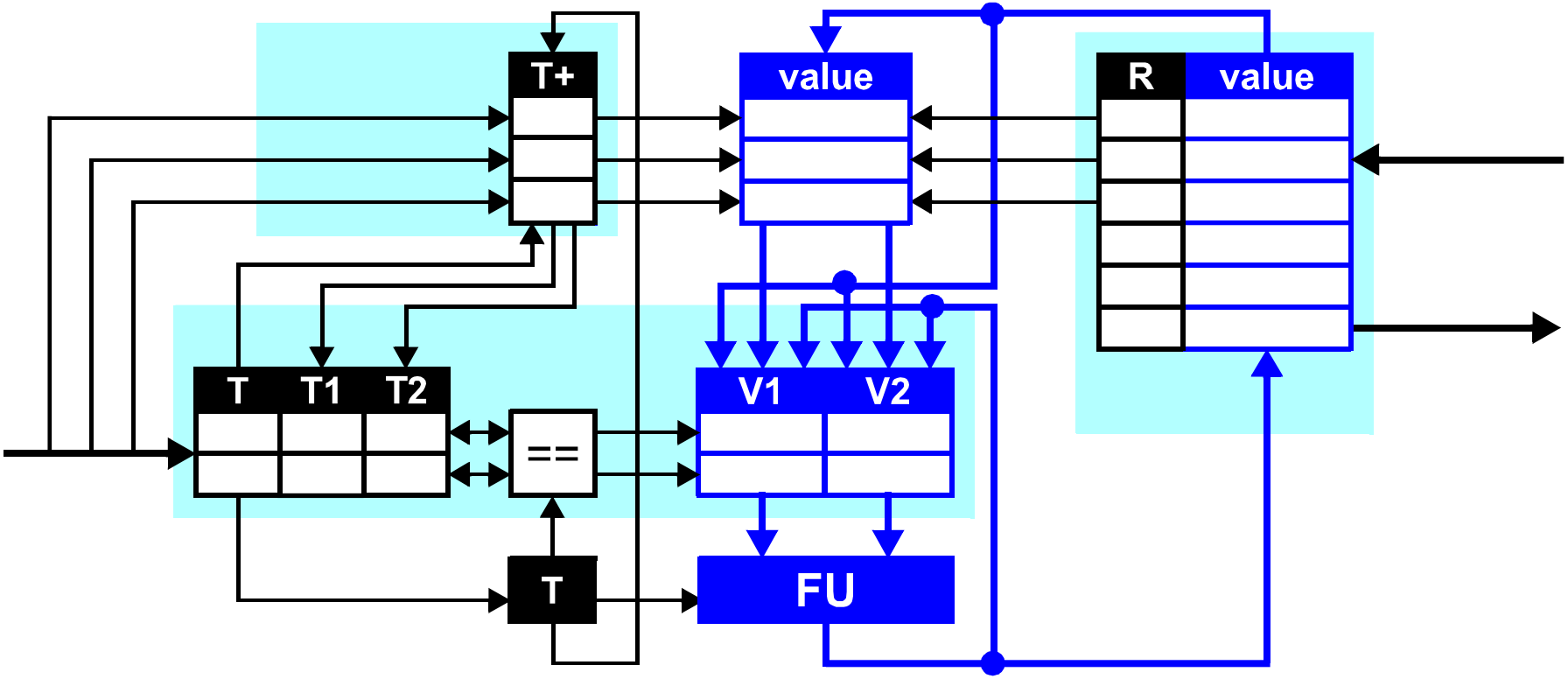
P6 Performance and Organization

- **Performance**
  - Generally the same performance as regular Tomasulo

- **Organization**
  - Popular design for a while

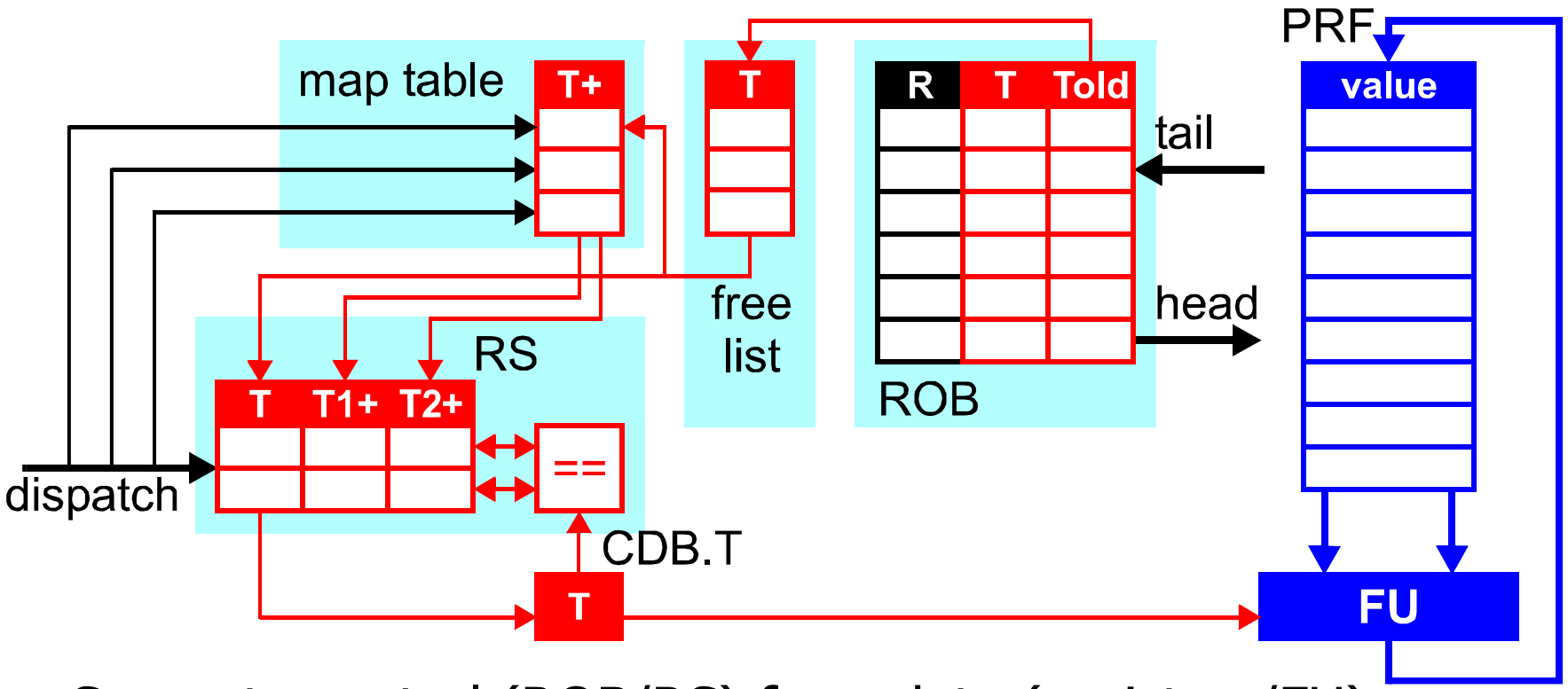
    \+ Easy (relatively) to implement correctly
  - To recover, just zero out everything
    - Examples: PentiumPro, PowerPC, AMD K6

Problems with P6



– Problems for high performance implementations
  - Too much value movement (regfile/ROB→RS, ROB→regfile)
  - Too many muxes/buses (values from too many places)
  - RS mixes values with control tags (long data paths)

Alternative Implementation: MIPS R10K



- Separate control (ROB/RS) from data (registers/FU)
  - One big **physical register file** (PRF) holds all data → no copies
  - ROB and RS used only for control and tags → small
  - PRF close to FUs, everything else is on the side

R10K Register Renaming

- No architectural register file!
- **Physical register file holds all values**
  - #physical registers > #architectural registers
  - Map architectural registers to physical registers
  - Removes WAW&WAR hazards (physical registers replace RS copies)
- Register status table replaced by **register map table**
  - Mappings cannot be 0 (there is no architectural register file)
- **Free list** keeps track of unallocated physical registers
  - ROB responsible for returning physical registers to free list
- **Conceptually:** true register renaming

R10K Pipeline

- Pipeline structure: IF, **DS, IS,** EX, **CM, RT**
  - **DS (dispatch)**
    - (RS or ROB or MOB full or no physical registers) ? (stall) :
      1. Allocate RS and ROB entries AND physical register
  - **IS (issue)**
    - If RAW ? (stall) : (read physical registers)
  - **EX (execute)**
    - Execute then free RS
  - **CM (completion)**
    - Wait for CDB
      1. Broadcast tag on CDB
      2. Write-back destination register
      3. Mark ROB entry complete

R10K Pipeline

- Pipeline structure: IF, **DS, IS,** EX, **CM, RT**
  - **RT (retire, commit, graduate)**
    - (ROB head not complete) ? (stall) :
    1. Handle any exceptions
    2. Write MOB head to D$ (if store)
    3. Free ROB/MOB entries
    4. Free previous physical register

R10K Data Structures

	ROB + MOB							
h/t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)						
	2	mulf f4,f0,f2						
	3	stf f4,Z(r1)						
	4	addi r1,r1,#4						
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#1+
f2	PR#2+
f4	PR#3+
r1	PR#4+

CDB
T

Free list
PR#5, PR#6, PR#7, PR#8

Must have mapping

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	No				
3	store	No				
4	FP1	No				
5	FP2	No				

Told for freeing old phys. regs.

Only tags in RS and CDB
No values, but need ready bits

R10K Example: Cycle 1

	ROB + MOB							
h/t	#	Instruction	T	Told	addr	IS	EX	CM
ht	1	ldf f0,X(r1)	PR#5	PR#1	&X[0]			
	2	mulf f4,f0,f2						
	3	stf f4,Z(r1)						
	4	addi r1,r1,#4						
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#5
f2	PR#2+
f4	PR#3+
r1	PR#4+

Free list
~~PR#5~~, PR#6, PR#7, PR#8

CDB
T

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	Yes	ldf	PR#5		PR#4+
3	store	No				
4	FP1	No				
5	FP2	No				

Remember old phys. reg. mapped to f0

Allocate new phys. reg. to f0

R10K Example: Cycle 2

	ROB + MOB							
h/ t	#	Instruction	T	Told	addr	IS	EX	CM
h	1	ldf f0,X(r1)	PR#5	PR#1	&X[0]	2		
t	2	mulf f4,f0,f2	PR#6	PR#3				
	3	stf f4,Z(r1)						
	4	addi r1,r1,#4						
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#5
f2	PR#2+
f4	PR#6
r1	PR#4+

Free list
~~PR#6~~, PR#7, PR#8

CDB
T

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	Yes	ldf	PR#5		PR#4+
3	store	No				
4	FP1	Yes	mulf	PR#6	PR#5	PR#2+
5	FP2	No				

Remember old phys. reg. mapped to f4

Allocate new phys. reg. to f4

R10K Example: Cycle 3

	ROB + MOB							
h/ t	#	Instruction	T	Told	addr	IS	EX	CM
h	1	ldf f0,X(r1)	PR#5	PR#1	&X[0]	2	3	
	2	mulf f4,f0,f2	PR#6	PR#3				
t	3	stf f4,Z(r1)			&Z[0]			
	4	addi r1,r1,#4						
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#5
f2	PR#2+
f4	PR#6
r1	PR#4+

CDB
T

Free list
PR#7, PR#8

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	No				
3	store	Yes	stf		PR#6	PR#4+
4	FP1	Yes	mulf	PR#6	PR#5	PR#2+
5	FP2	No				

Stall due to RAW

Free RS

Allocate RS

R10K Example: Cycle 4

	ROB + MOB							
h/ t	#	Instruction	T	Told	addr	IS	EX	CM
h	1	ldf f0,X(r1)	PR#5	PR#1	&X[0]	2	3	4
	2	mulf f4,f0,f2	PR#6	PR#3		4		
	3	stf f4,Z(r1)			&Z[0]			
t	4	addi r1,r1,#4	PR#7	PR#4				
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#5+
f2	PR#2+
f4	PR#6
r1	PR#7

Free list
~~PR#7~~, PR#8

CDB
T
PR#5

ldf completed
Set ready bit, CDB broadcast

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	Yes	addi	PR#7	PR#4+	
2	load	No				
3	store	Yes	stf		PR#6	PR#4+
4	FP1	Yes	mulf	PR#6	PR#5+	PR#2+
5	FP2	No				

Allocate RS

Match PR#5 tag from CDB and issue

R10K Example: Cycle 5

	ROB + MOB							
h/t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)	PR#5	PR#1	&X[0]	2	3	4
h	2	mulf f4,f0,f2	PR#6	PR#3		4	5	
	3	stf f4,Z(r1)			&Z[0]			
	4	addi r1,r1,#4	PR#7	PR#4		5		
t	5	ldf f0,X(r1)	PR#8	PR#5				
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#8
f2	PR#2+
f4	PR#6
r1	PR#7

CDB
T

Free list
PR#1
~~PR#8~~

ldf retires
Return PR#1 to free list
No one will read it

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	Yes	addi	PR#7	PR#4+	
2	load	Yes	ldf	PR#8		PR#7
3	store	Yes	stf		PR#6	PR#4+
4	FP1	No				
5	FP2	No				

Allocate RS

Free RS (pipelined *unit)

R10K Example: Cycle 6

	ROB + MOB							
h/ t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)				2	3	4
h	2	mulf f4,f0,f2	PR#6	PR#3		4	5+	
	3	stf f4,Z(r1)			&Z[0]			
	4	addi r1,r1,#4	PR#7	PR#4		5	6	
	5	ldf f0,X(r1)	PR#8	PR#5				
t	6	mulf f4,f0,f2	PR#1	PR#6				
	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#8
f2	PR#2+
f4	PR#1
r1	PR#7

CDB
T

Free list
~~PR#1~~

IS Stall: ldf waits on r1

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	Yes	ldf	PR#8		PR#7
3	store	Yes	stf		PR#6	PR#4+
4	FP1	No				
5	FP2	Yes	mulf	PR#1	PR#8	PR#2+

Allocate RS

R10K Example: Cycle 7

	ROB + MOB							
h/ t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)				2	3	4
h	2	mulf f4,f0,f2	PR#6	PR#3		4	5+	
	3	stf f4,Z(r1)			&Z[0]			
	4	addi r1,r1,#4	PR#7	PR#4		5	6	7
	5	ldf f0,X(r1)	PR#8	PR#5		7		
	6	mulf f4,f0,f2	PR#1	PR#6				
t	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#8
f2	PR#2+
f4	PR#1
r1	PR#7+

Free list

CDB
T
PR#7

addi finished
Set ready bit, CDB broadcast

DS Stall: structural hazard

Match PR#7 tag from CDB and issue

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	Yes	ldf	PR#8		PR#7+
3	store	Yes	stf		PR#6	PR#4+
4	FP1	No				
5	FP2	Yes	mulf	PR#1	PR#8	PR#2+

R10K Example: Cycle 8

	ROB + MOB							
h/t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)				2	3	4
h	2	mulf f4,f0,f2	PR#6	PR#3		4	5+	8
	3	stf f4,Z(r1)			&Z[0]	8		
	4	addi r1,r1,#4	PR#7	PR#4		5	6	7
	5	ldf f0,X(r1)	PR#8	PR#5		7	8	
	6	mulf f4,f0,f2	PR#1	PR#6				
t	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#8
f2	PR#2+
f4	PR#1
r1	PR#7+

CDB
T
PR#6

Free list

RT Stall: addi is not head

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	No				
3	store	Yes	stf		PR#6+	PR#4+
4	FP1	No				
5	FP2	Yes	mulf	PR#1	PR#8	PR#2+

mulf finished
CDB broadcast

Free RS

Match PR#6 tag from CDB and issue

R10K Example: Cycle 9

	ROB + MOB							
h/t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)				2	3	4
	2	mulf f4,f0,f2	PR#6	PR#3		4	5+	8
h	3	stf f4,Z(r1)			&Z[0]	8	9	
	4	addi r1,r1,#4	PR#7	PR#4		5	6	7
	5	ldf f0,X(r1)	PR#8	PR#5		7	8	9
	6	mulf f4,f0,f2	PR#1	PR#6		9		
t	7	stf f4,Z(r1)			&Z[1]			

Map Table	
Reg	T+
f0	PR#8+
f2	PR#2+
f4	PR#1
r1	PR#7+

Free list
PR#3

CDB
T
PR#8

ldf finished, CDB broadcast

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	No				
3	store	Yes	stf		PR#1	PR#7+
4	FP1	No				
5	FP2	Yes	mulf	PR#1	PR#8+	PR#2+

mulf retires
Return PR#3 to free list
No one will read it

Free RS, then allocate,
Get r1 from PRF

Match PR#8 tag and get
from CDB

R10K Example: Cycle 10

	ROB + MOB							
h/ t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)				2	3	4
	2	mulf f4,f0,f2				4	5+	8
h	3	stf f4,Z(r1)			&Z[0]	8	9	10
	4	addi r1,r1,#4	PR#7	PR#4		5	6	7
	5	ldf f0,X(r1)	PR#8	PR#5		7	8	9
	6	mulf f4,f0,f2	PR#1	PR#6		9	10	
t	7	stf f4,Z(r1)			&Z[1]			

Map Table	
Reg	T+
f0	PR#8+
f2	PR#2+
f4	PR#1
r1	PR#7+

CDB
T

Free list
PR#3

RT Stall

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	No				
3	store	Yes	stf		PR#1	PR#7+
4	FP1	No				
5	FP2	No				

Free RS

R10K Example: Cycle 11

	ROB + MOB							
h/ t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)				2	3	4
	2	mulf f4,f0,f2				4	5+	8
h	3	stf f4,Z(r1)			&Z[0]	8	9	10
	4	addi r1,r1,#4	PR#7	PR#4		5	6	7
	5	ldf f0,X(r1)	PR#8	PR#5		7	8	9
t	6	mulf f4,f0,f2	PR#1	PR#6		9	10	
	7	stf f4,Z(r1)			&Z[1]			

Map Table	
Reg	T+
f0	PR#8+
f2	PR#2+
f4	PR#1
r1	PR#7+

Free list
PR#3

CDB
T

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	No				
3	store	No				
4	FP1	No				
5	FP2	No				

Interrupt on stf in RT

Undo stf

1. Free stf RS
2. Free ROB#7
3. No registers to restore

R10K Example: Cycle 12

	ROB + MOB							
h/t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)				2	3	4
	2	mulf f4,f0,f2				4	5+	8
h	3	stf f4,Z(r1)			&Z[0]	8	9	10
	4	addi r1,r1,#4	PR#7	PR#4		5	6	7
t	5	ldf f0,X(r1)	PR#8	PR#5		7	8	9
	6	mulf f4,f0,f2	PR#1	PR#6		9	10	
	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#8
f2	PR#2+
f4	PR#1
r1	PR#7+

CDB
T

Free list
PR#1, PR#3

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	No				
3	store	No				
4	FP1	No				
5	FP2	No				

Undo mulf

1. Free mulf RS
2. Free T (PR#1)
3. Restore f4 mapping with Told (PR#1)
4. Free ROB#6

R10K Example: Cycle 13

	ROB + MOB							
h/ t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)				2	3	4
	2	mulf f4,f0,f2				4	5+	8
h	3	stf f4,Z(r1)			&Z[0]	8	9	10
t	4	addi r1,r1,#4	PR#7	PR#4		5	6	7
	5	ldf f0,X(r1)	PR#8	PR#5		7	8	9
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#5+
f2	PR#2+
f4	PR#6+
r1	PR#7+

CDB
T

Free list
PR#1, PR#3, PR#8

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	No				
3	store	No				
4	FP1	No				
5	FP2	No				

Undo ldf

1. Free ldf RS
2. Free T (PR#8)
3. Restore f0 mapping with Told (PR#5)
4. Free ROB#5

R10K Example: Cycle 14

	ROB + MOB							
h/t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)				2	3	4
	2	mulf f4,f0,f2				4	5+	8
h/t	3	stf f4,Z(r1)			&Z[0]	8	9	10
	4	addi r1,r1,#4	PR#7	PR#4		5	6	7
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#5+
f2	PR#2+
f4	PR#6+
r1	PR#4+

CDB
T

Free list
PR#1, PR#3, PR#7,PR#8

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	No				
3	store	No				
4	FP1	No				
5	FP2	No				

Undo addi

1. Free addi RS
2. Free T (PR#7)
3. Restore f0 mapping with Told (PR#4)
4. Free ROB#5

R10K Example: Cycle 15

	ROB + MOB							
h/t	#	Instruction	T	Told	addr	IS	EX	CM
	1	ldf f0,X(r1)				2	3	4
	2	mulf f4,f0,f2				4	5+	8
h/t	3	stf f4,Z(r1)						
	4	addi r1,r1,#4						
	5	ldf f0,X(r1)						
	6	mulf f4,f0,f2						
	7	stf f4,Z(r1)						

Map Table	
Reg	T+
f0	PR#5+
f2	PR#2+
f4	PR#6+
r1	PR#4+

CDB
T

Free list
PR#1, PR#3, PR#7,PR#8

Reservation Stations						
T	FU	Busy	op	T	T1+	T2+
1	ALU	No				
2	load	No				
3	store	No				
4	FP1	No				
5	FP2	No				

Undo stf

1. Free addi RS
2. Free ROB#3
3. Restart insn

P6 vs. R10K

Features	P6	R10K
Storage: values	RF,ROB,RS	PRF
Register read	DS: RF/ROB→RS	IS:PRF→FU
Register write	RT: ROB→RF	CM:FU→PRF
Freeing	RT: Automatic with ROB	RT: Overwritten insn retires
Datapaths	RF/ROB→RS, RS→FU, FU→ROB, ROB→RF	PRF→FU, FU→PRF
Precise state	Simple: 0 all structures	Complex: rollback/checkpoint

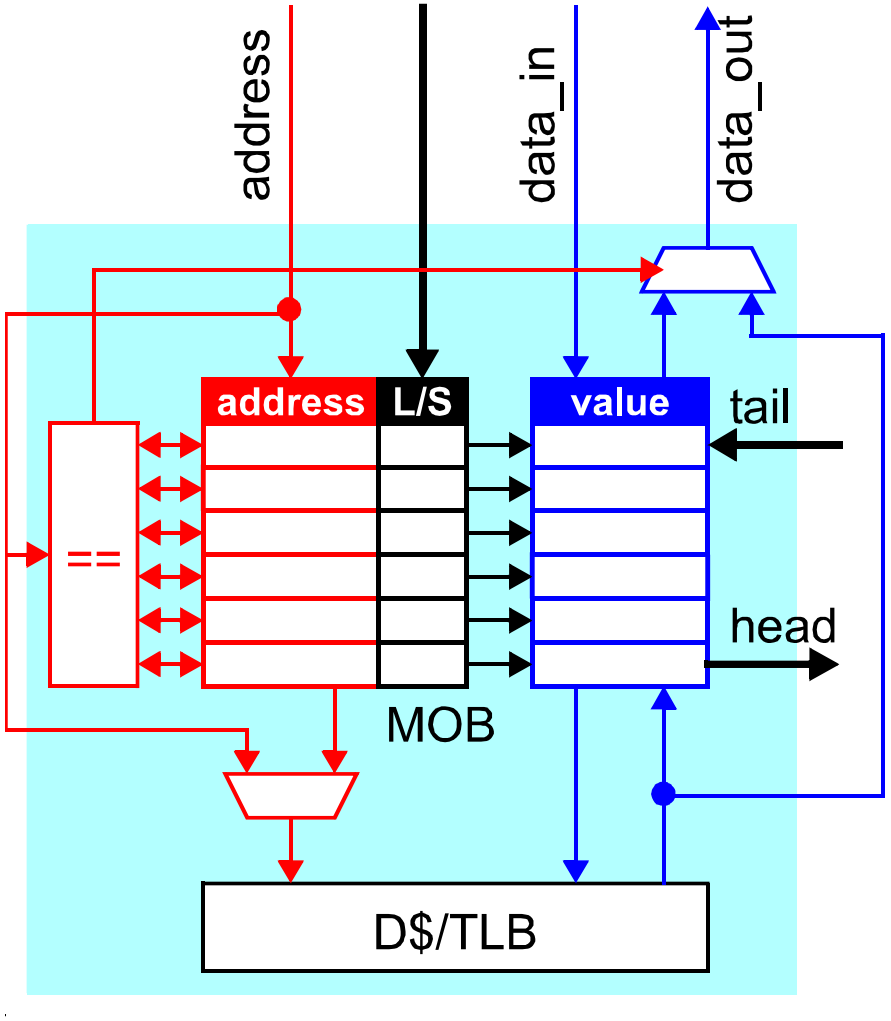
MOB: Load Scheduling

- All instructions except for loads are easy in Tomasulo
  - Register renaming captures all dependencies
  - Tags tell you exactly when you can execute

- Loads not so easy ☹
  - Must check for older active stores with same address
  - Register renaming doesn't tell you that
  - Want to allow load-store forwarding

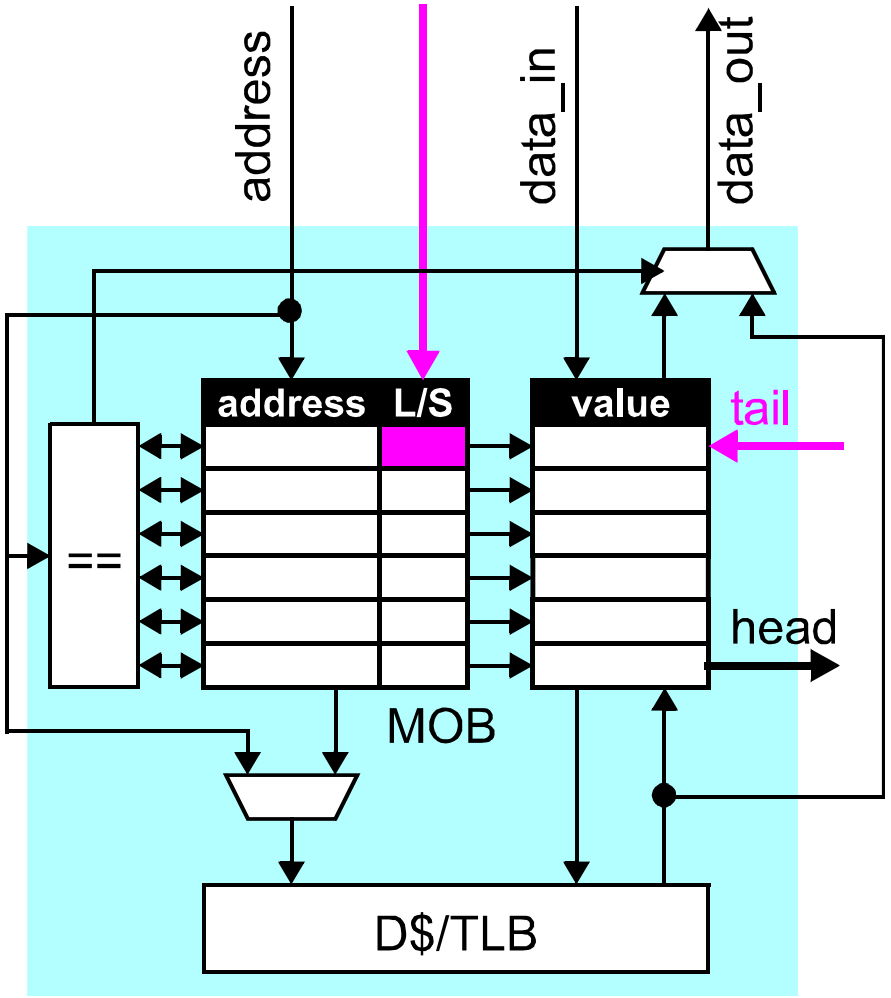
Let us focus on Data Memory FU

- MOB+D$= memory FU
  - Just like any other FU
  - 2 reg inputs
    - address, data_in
  - 1 reg output
    - data_out
- How does this work?



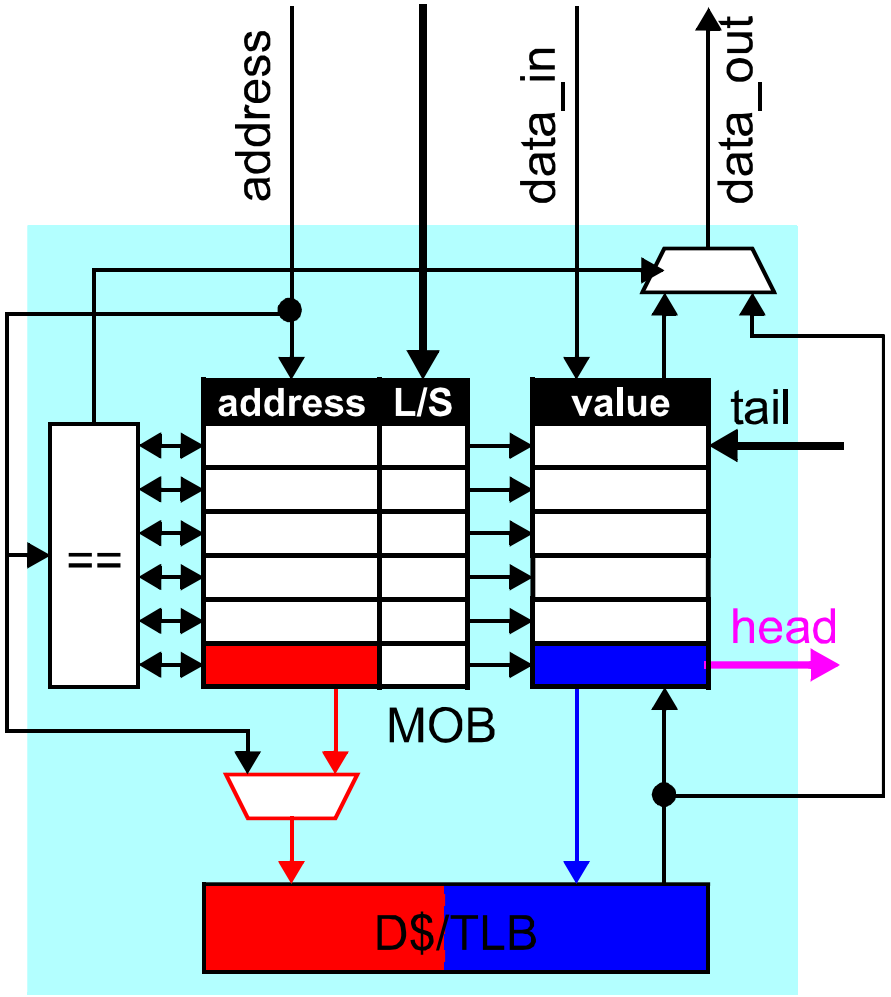
Store/Load Dispatch

- Allocate MOB entry (tail)
  - Indicate store/load
  - Remember MOB# in RS



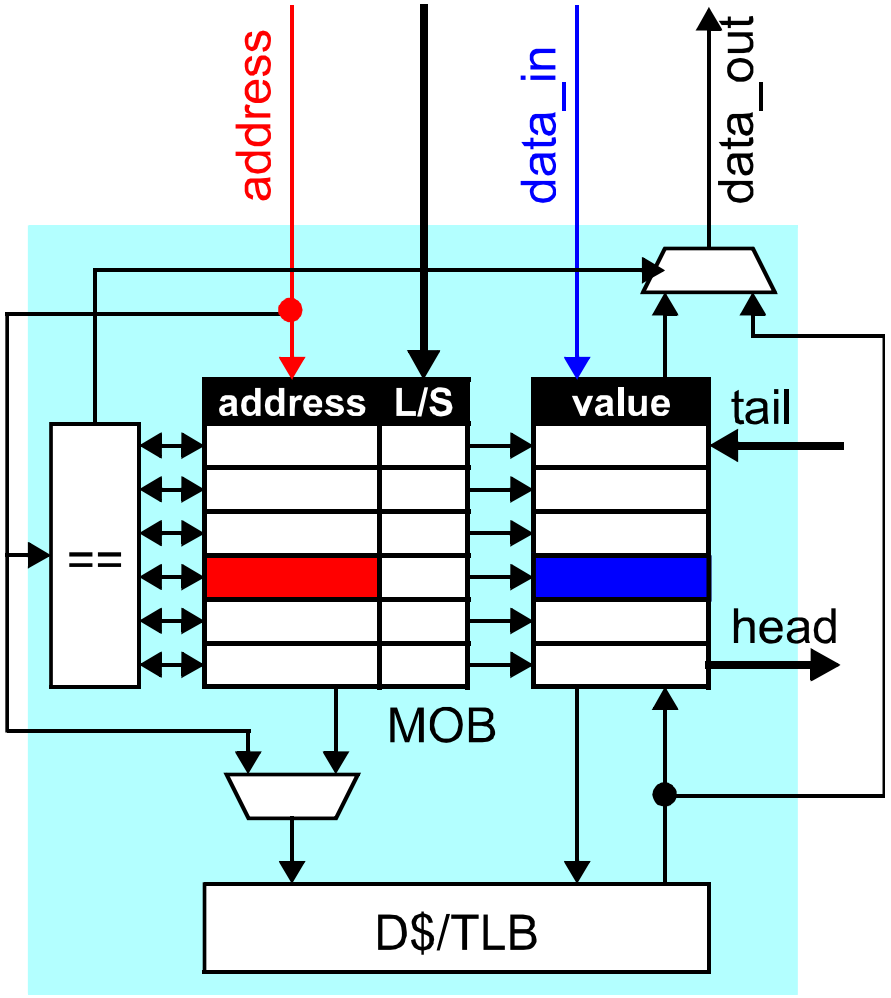
Store/Load Retire

- Free MOB entry (head)
- Load?
  - Done
- Store?
  - address+value to D$



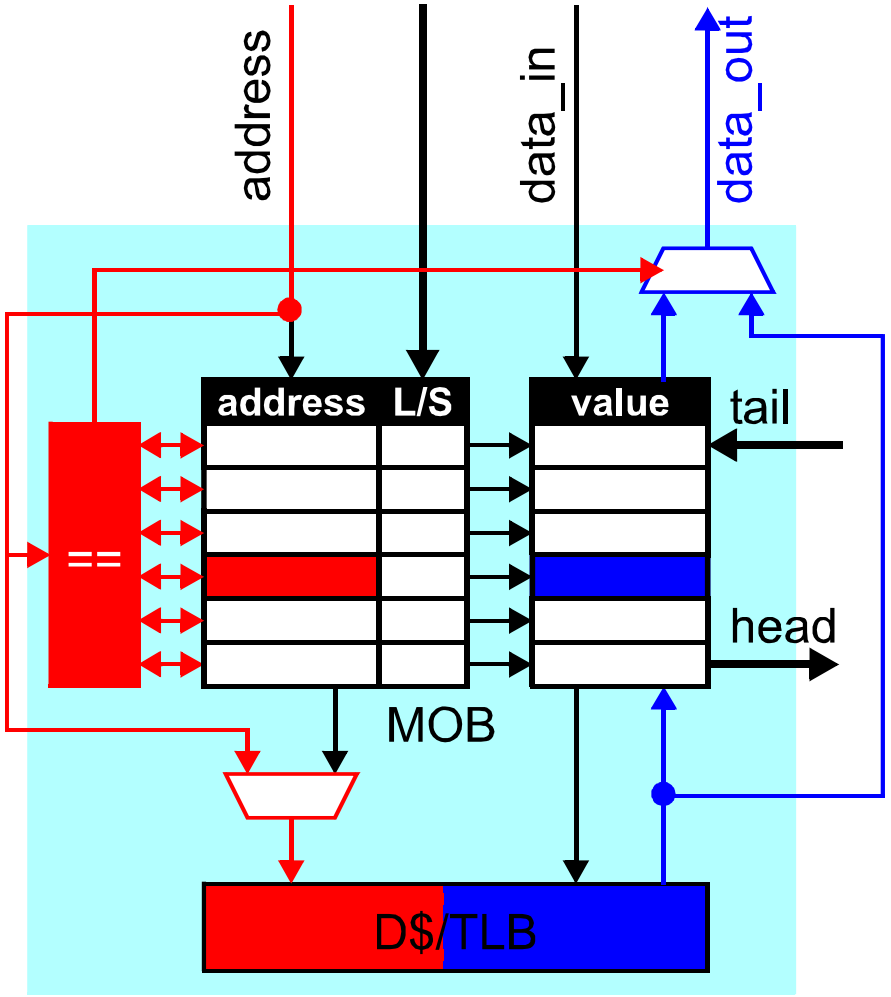
Store Execute

- address+value to MOB



Load Execute

- In parallel
  - Address to D$
    - Read value
  - Address to MOB
    - Compare to older store addresses



Load Execute

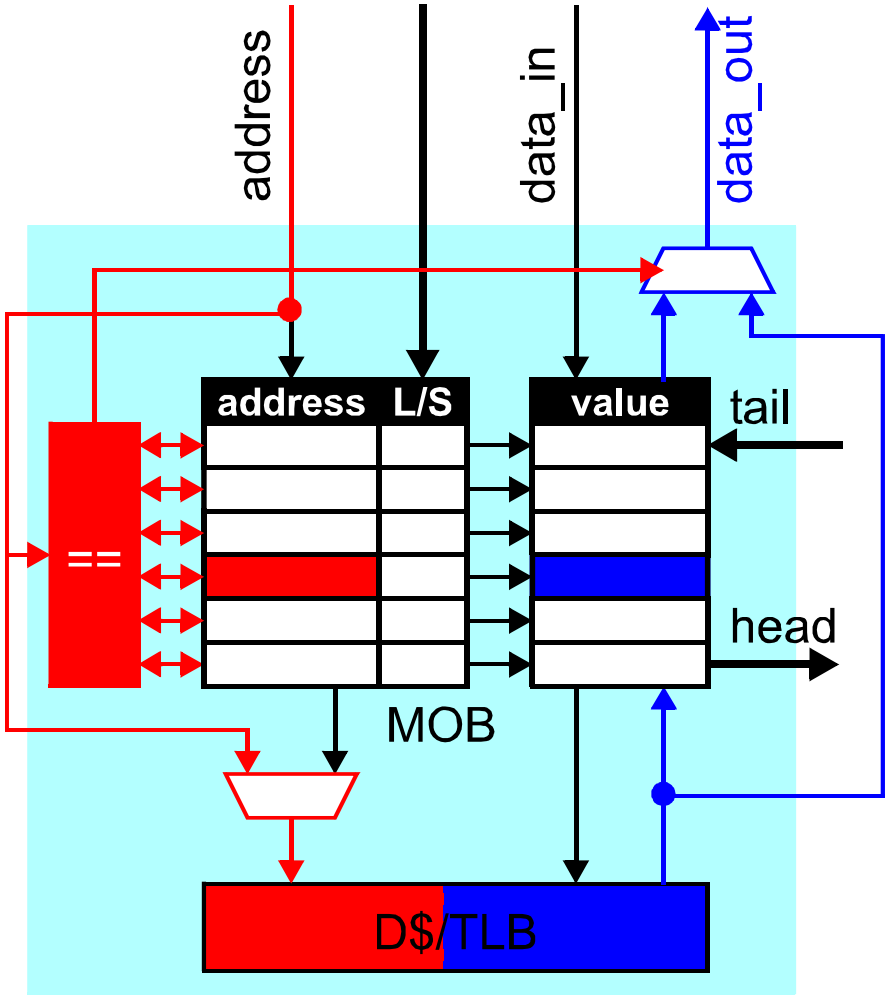
- No match in MOB

  ⇒ D$ value

- Match

  Has value ⇒ MOB value

  - Forward
  - Multiple matches? youngest matching store older than load



Lets Get Real: Pentium II vs. Pentium 4

	Pentium II	Pentium 4
Peak Clock	450Mhz (Xeon)	2.4 Ghz
Pipeline Stages	14	22
Branch Prediction	512 entry local	2K entry hybrid
BTB	512 entries	4K entries
I$	16KB	8KB I$ + 64KB T$
D$	16KB	256KB-2MB
Fetch width	16 bytes	3uops
Rename/Retire width	3uops	3uops
Execute width	5uops	7uops
Reservation Stations	20	60
ROB Size	40	128
Register Renaming	P6-style	MIPS R10K-style
Memory Disambiguation	Conservative	Predictor-based

Summary

- Dynamic scheduling + precise state + speculation
  - Re-order buffer (ROB)
  - WB ⇒ CM (out-of-order) + RT (in-order)
  - P6 vs. R10K
  - How is recovery done?

- Load scheduling using
  - MOB+D$, bypassing values from MOB